PROJECT REPORT

AI SOC

Using Wazuh + n8n + Gemini

An AI-assisted security alert retrieval, classification and summarization workflow

Prepared by

VIPIN KUMAR

LinkedIn: linkedin.com/in/vipin-kumar-530a55283

GitHub: github.com/vipinsky

Technical project documentation | November 2026

1. Executive Summary

This project demonstrates an AI-assisted Security Operations Center (SOC) workflow that connects Wazuh with n8n and Google Gemini. Wazuh provides the security alert data, n8n coordinates the automation, and Gemini helps turn retrieved alert records into a readable analyst-oriented summary. A chat-style SOC Assistant Console is shown as the user-facing interface for submitting questions and viewing responses.

The implementation is distributed across two virtual machines. The first hosts the Wazuh Manager, Dashboard and Indexer. The second hosts a Wazuh Agent and Docker, with n8n running in a Docker container. An n8n webhook receives a request, the AI Agent interprets it, an HTTP Request tool queries the Wazuh index, and Gemini generates a concise explanation based on the returned data. The workflow then returns the response through a webhook response node.

What this project is designed to achieve

Reduce the effort involved in manually searching security alerts by giving an analyst a natural-language way to request alert information and receive a structured summary. The AI output is an investigation aid; it is not a replacement for validating evidence in Wazuh.

Project Snapshot

Item	Details
Project name	AI SOC Using Wazuh + n8n + Gemini
Project category	Security Operations / SIEM automation / AI-assisted alert triage
Core platforms	Wazuh, n8n, Docker, Google Gemini
Infrastructure	Two virtual machines; n8n deployed in a Docker container on VM 2
Primary workflow	Webhook -> AI Agent -> Wazuh HTTP query -> Gemini summary -> Webhook response
User interaction	Chat-style SOC Assistant Console shown in the supplied screenshots
Project author	Vipin Kumar

Problem Statement

SOC analysts often need to review many security events, identify which alerts deserve attention, understand repeated rule activity, and communicate findings clearly. Searching raw event records can take time, particularly when an analyst needs a quick overview for a specific time range or alert category.

Project Objectives

- Connect a Wazuh-based monitoring environment to an automation workflow through n8n.
- Accept alert-analysis requests through a webhook-driven assistant interface.
- Retrieve alert records from the Wazuh index using an HTTP Request node configured with POST and Wazuh authentication.
- Classify alerts using the Wazuh rule.level ranges defined in the project prompt.
- Group alerts by severity, type and frequency, and identify repeated rules or notable patterns.
- Use Gemini to produce a clear, concise, human-readable summary of the retrieved alert data.

2. System Architecture and Data Flow

The project separates the SIEM components from the automation layer. This makes the roles of the two virtual machines easy to understand and keeps n8n deployment independent from the Wazuh server services.

Infrastructure Layout

Environment	Installed components	Role in the project
VM 1 - Wazuh server	Wazuh Manager, Wazuh Dashboard, Wazuh Indexer	Receives/handles endpoint security events, stores indexed alert data, and provides the monitoring interface.
VM 2 - automation host	Wazuh Agent, Docker, n8n container	Provides an endpoint monitored by Wazuh and hosts the automation workflow that receives requests and coordinates AI-assisted analysis.
External AI service	Google Gemini API / Gemini model node	Generates a readable summary from alert information supplied by the workflow.
User-facing layer	SOC Assistant Console shown in the screenshots	Allows an analyst to submit a natural-language alert question and review the returned response.

End-to-End Request Flow

Stage	What happens
1. Analyst asks a question	A user enters a request in the SOC Assistant Console, such as asking for the latest high-severity alerts.
2. Webhook receives the request	The request is sent to the n8n webhook that acts as the workflow entry point.
3. AI Agent interprets the request	The n8n AI Agent uses the configured system instructions to understand the requested alert scope and analysis.
4. HTTP Request retrieves Wazuh data	The agent uses the HTTP Request tool, configured with POST, the Wazuh URL and authentication, to query alert records.
5. Alert data is organized	The instructions tell the agent to map rule.level to severity and group results by severity, type and frequency.
6. Gemini creates the summary	The Gemini model is used to produce a concise explanation and call out repeated rules, notable patterns or anomalies.
7. Workflow returns the response	The webhook response node sends the generated answer back to the requesting interface.

Key Design Choice

The workflow combines deterministic instructions (the rule-level severity mapping and default query scope) with AI-generated natural-language explanations. The severity bands are explicitly defined in the system prompt; Gemini is used for summarization and interpretation of the returned alert data.

3. Workflow Configuration and Alert Analysis Logic

n8n Workflow Components

Component	Purpose in this implementation
Webhook	Receives the incoming user request and starts the workflow.
AI Agent	Coordinates the requested alert investigation using the system message and connected tools.
HTTP Request tool	Uses the POST method to request alert data from the configured Wazuh URL with Wazuh authentication.
Gemini model	Creates the human-readable summary from the retrieved alert data.
Simple Memory	A memory component is connected to the AI Agent in the workflow to support context handling.
Webhook Response	Returns the workflow result to the caller / assistant interface.

Severity Classification

The system message instructs the SOC AI Assistant to convert the numeric Wazuh rule.level value into one of four readable severity categories:

Wazuh rule.level	Severity label	How to interpret it
0-3	Low	Lower-severity events for routine monitoring and context.
4-7	Medium	Events that may require review based on context and repetition.
8-11	High	Higher-priority events that should receive closer analyst attention.
12 and above	Critical	Highest-priority category in the project mapping.

Analysis Instructions Provided to the AI Agent

- Query the Wazuh index using a supplied alert count or time range; if none is supplied, use the latest 100 alerts from the past 24 hours.
- Map each returned rule.level value to the corresponding severity label.
- Group and categorize alerts by severity, type and frequency.
- Use Gemini to generate a clear, concise summary of the alert data.
- Highlight notable patterns, repeated rules or anomalies.

4. Implementation Evidence

The following figures are taken from the screenshots included in the supplied implementation document. They illustrate the workflow editor, the Wazuh monitoring dashboard and the alert records available for analysis.

4.1 n8n Workflow Editor

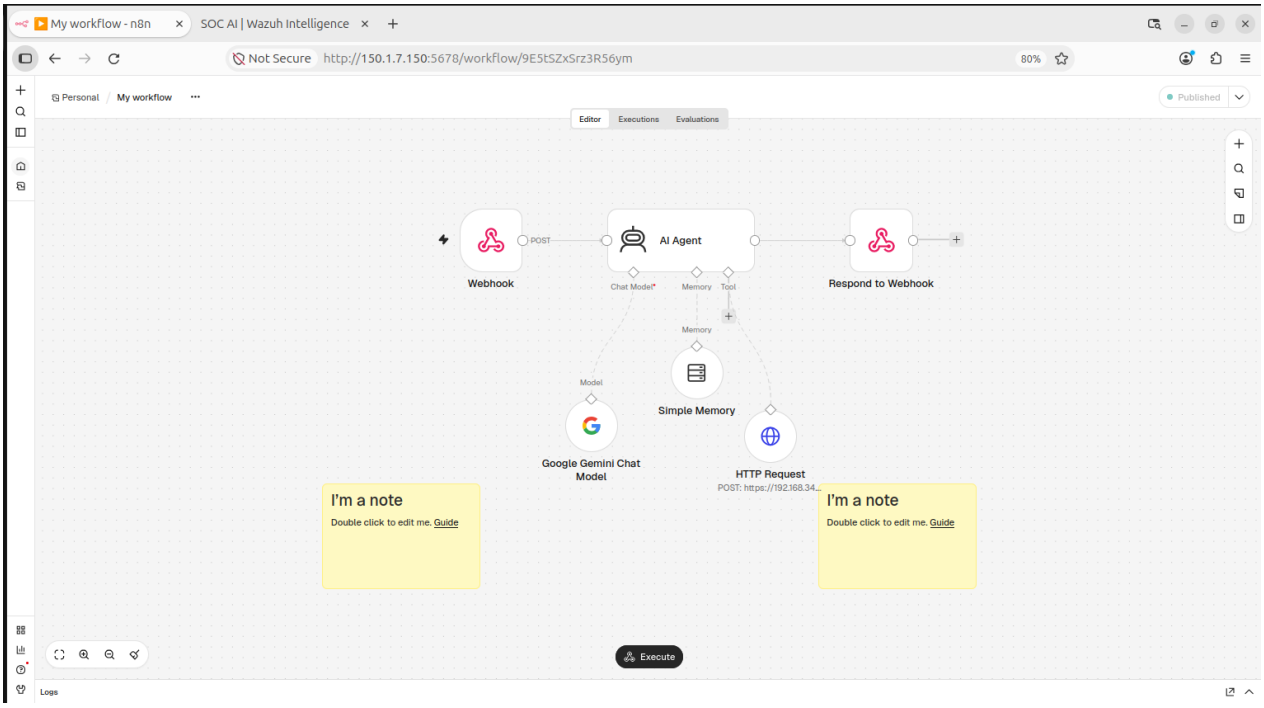


Figure 1. n8n workflow editor showing the connected webhook, AI Agent, Gemini, memory, HTTP Request and response components.

The workflow screenshot supports the documented design: the webhook is used as the entry point, the AI Agent is connected to Gemini and Simple Memory, and the HTTP Request capability is available to retrieve data from Wazuh. The final response is returned through the webhook response step.

4.2 Wazuh Monitoring Dashboard

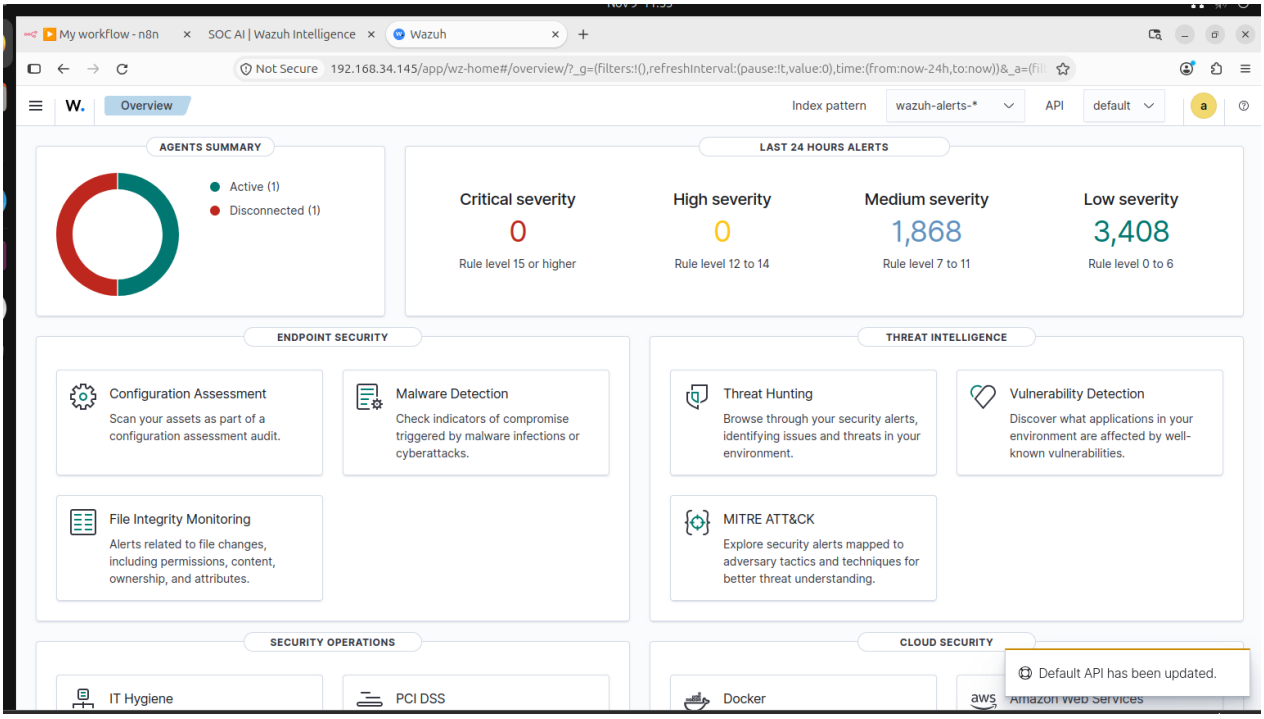


Figure 2. Wazuh Dashboard overview showing the security monitoring interface and alert-severity summary area.

The dashboard view provides the monitoring context for the project. The screenshot includes the alert summary area and Wazuh security modules. The counts shown are time-specific to the captured screen and are not presented here as current or long-term performance metrics.

4.3 Wazuh Alert Records

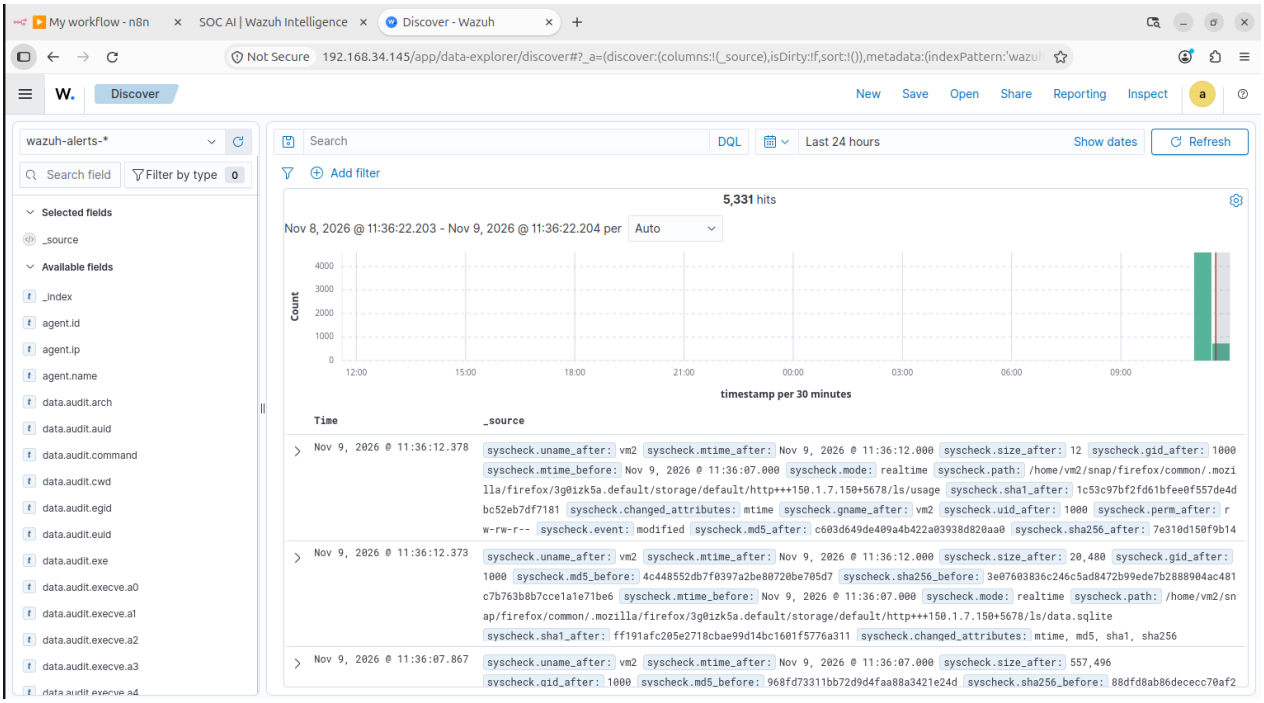


Figure 3. Wazuh alert/event view showing indexed records that can be queried by the automation workflow.

This view illustrates the underlying alert data source. The automation is intended to retrieve records from the Wazuh index and convert the returned event information into grouped, readable findings.

4.4 SOC Assistant Console

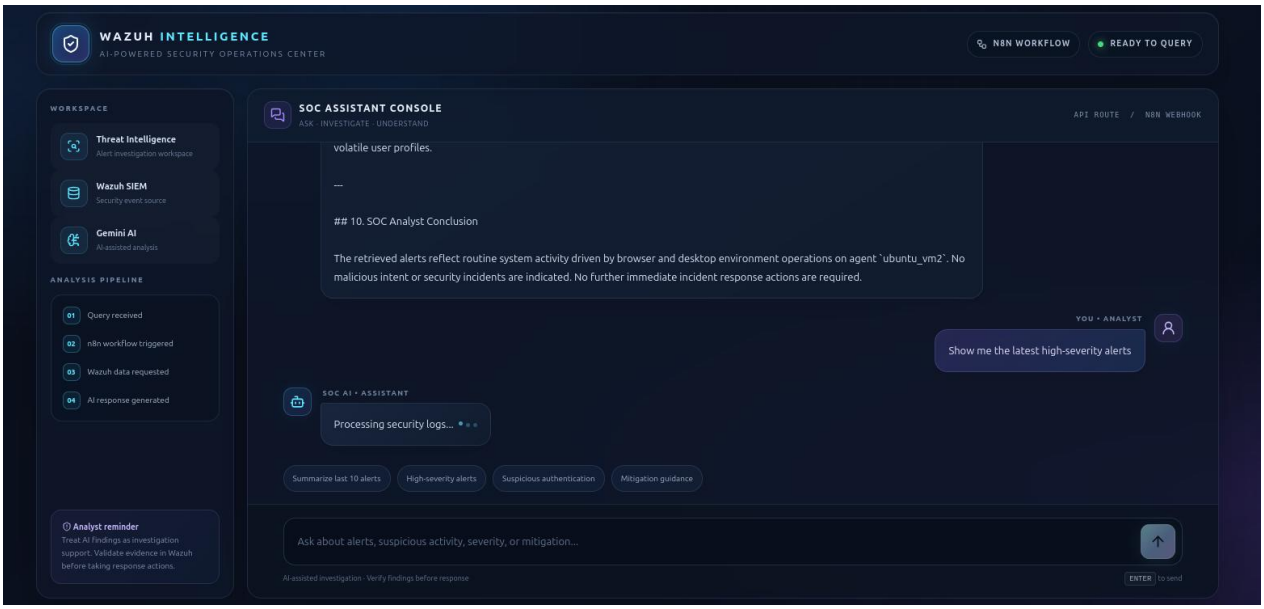


Figure 4. Chat-style SOC Assistant Console showing a user request for high-severity alerts and the workflow status.

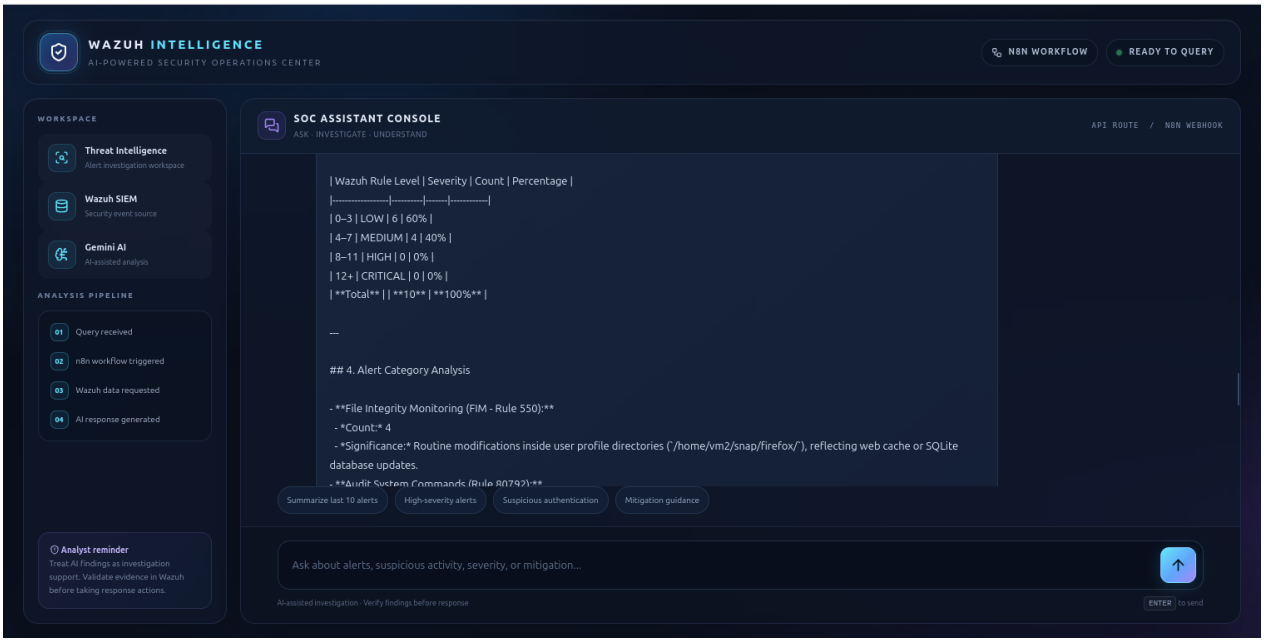


Figure 5. Assistant output showing the severity breakdown and the beginning of an alert-category analysis.

The supplied console screenshots show the user-facing interaction and an example generated analysis. The interface also displays a simple pipeline: query received, n8n workflow triggered, Wazuh data requested, and AI response generated. This is useful evidence of the intended request-to-response experience.

Sample Analysis Displayed in the Screenshot

One supplied assistant-console screenshot shows a sample severity breakdown of 10 retrieved alerts: 6 Low alerts (60%), 4 Medium alerts (40%), and 0 High or Critical alerts. It also begins a File Integrity Monitoring (FIM) category analysis, showing rule 550 with a count of 4 and describing routine browser-profile changes as the apparent context. This is an example output shown in the project evidence, not a benchmark, a guarantee of detection accuracy, or proof that all future activity will be benign.

5. Demonstrated Capabilities, Scope and Validation

Capabilities Demonstrated by the Supplied Evidence

- Two-VM separation between Wazuh services and the automation host.
- Docker-based deployment of n8n on the second virtual machine.
- Webhook-based workflow entry and response handling.
- An AI Agent configured with SOC-focused instructions and connected analysis tools.
- HTTP POST configuration to retrieve Wazuh alert data using supplied authentication.
- Gemini API key creation and verification as recorded in the source notes.
- Rule-level severity mapping, alert grouping instructions, and a default last-100-alerts / past-24-hours query scope.
- A chat-style assistant screen with an example alert summary and category analysis.

What the Current Project Evidence Does Not Establish

The supplied notes and screenshots describe the architecture and show example screens, but they do not include enough detail to independently verify every production-readiness or performance claim. The following items are not specified in the source material:

- The exact Wazuh index endpoint, index pattern, query body, field list, and pagination behavior.
- The exact credential-storage method, webhook authentication method, TLS configuration, and network access restrictions.
- Formal test cases covering known benign and malicious events, false-positive rates, detection coverage, or response-time measurements.
- Error handling, retry logic, API rate-limit handling, audit logging, and monitoring of failed workflow executions.
- A workflow export, source code, deployment scripts, or a reproducible installation guide.

Recommended Security and Reliability Improvements

- Secret management: store Wazuh credentials and the Gemini API key in n8n Credentials or an approved secret manager; do not hard-code them in nodes, screenshots, or GitHub files. Rotate any key that may have been exposed.
- Webhook protection: use HTTPS, authentication or signed requests, restrict network exposure, and apply rate limits and input validation.
- Least privilege: use a dedicated read-only Wazuh account scoped to only the required alert indices; protect and validate TLS certificates.
- Data privacy: review what alert fields are sent to Gemini, remove unnecessary personal or sensitive data, and confirm the organization is comfortable with the chosen AI data-processing arrangement.
- AI safety: treat log content as untrusted evidence, not instructions. Keep the agent from following commands embedded in logs, and require an analyst to validate findings against Wazuh before taking action.
- Operational resilience and testing: add timeouts, appropriate retries, clear error responses, pagination limits and workflow execution logs; validate with known benign events and controlled test alerts, then record accuracy, missed alerts, false positives, latency and API failures.

7. Conclusion

The AI SOC Using Wazuh + n8n + Gemini project demonstrates how SIEM alert data can be connected to a workflow automation platform and presented through an AI-assisted interface. The two-VM lab separates Wazuh monitoring services from the n8n automation host, while the webhook, AI Agent, HTTP Request, Gemini model, Simple Memory and webhook response components form the request-to-summary workflow.

The project defines a clear severity classification based on Wazuh rule.level, includes a default alert retrieval scope, and instructs the AI Agent to organize findings by severity, type and frequency. The supplied screenshots also show the Wazuh monitoring interface, alert records, the n8n workflow and a sample SOC Assistant response. Together, these elements provide a practical demonstration of SIEM integration, automation and AI-assisted alert understanding.

The most important next step is to strengthen the project with security controls and repeatable validation: protect credentials and webhooks, restrict access, confirm data handling with the AI service, test against known alert scenarios, and document accuracy and reliability. With those improvements and a public workflow/deployment guide, this lab can become a stronger portfolio demonstration of practical SOC engineering skills.

Final takeaway

This project shows the ability to connect security monitoring data with automation and AI to make alert information easier to query and understand - while keeping analyst verification central to the investigation process.